

MudClient Lua 腳本撰寫指南

適用版本 : mudclient-rs (Rust + Lua 架構)

最後更新 : 2026-02-20

目錄

- 1. [目錄結構與載入順序](#)
- 2. [腳本範本](#)
- 3. [顯式依賴宣告 \(必要\)](#)
- 4. [模組介紹](#)
- 5. [Hook 機制](#)
- 6. [全域命名空間規範](#)
- 7. [Timer 與 run_id 防競態](#)
- 8. [reload\(\) 標準實作](#)
- 9. [腳本載入流程一覽](#)

目錄結構與載入順序

```
scripts/
├── modules/          # ★ Phase 1 : 優先載入 (依字母排序)
│   ├── MudCombat.lua
│   ├── MudExplorer.lua
│   ├── MudLoot.lua
│   ├── MudMapper.lua
│   ├── MudNav.lua
│   └── MudUtils.lua
├── *.lua             # Phase 2 : 頂層腳本載入 (依字母排序)
└── tests/            # 測試腳本 (不自動載入)
```

重要 : Rust 層保證 `modules/` 在所有頂層腳本之前載入。
即便如此，每個腳本仍應顯式宣告依賴，原因見下節。

腳本範本

以下是新腳本的標準起手式，請直接複製修改：

```
-- =====
-- MyScript - 說明
-- =====
-- 使用: /lua MyScript.start()
-- 停止: /lua MyScript.stop()
-- =====

-- ===== 依賴宣告 (必要) =====
```

```

local function require_module(name)
    local paths = { "scripts.modules." .. name, "modules." .. name, name }
    for _, p in ipairs(paths) do
        local ok, res = pcall(require, p)
        if ok then return res end
    end
    error("MyScript cannot load dependency: " .. name)
end

local MudUtils = require_module("MudUtils")
-- local MudNav = require_module("MudNav") -- 按需取消
-- local MudCombat = require_module("MudCombat") -- 按需取消

-- ===== 全域表格 =====
_G.MyScript = _G.MyScript or {}

-- ===== 狀態 =====
_G.MyScript.state = {
    running = false,
    run_id = 0,
}

-- ===== Hook 註冊 =====
MudUtils.register_hook("MyScript", function(line, clean_line)
    _G.MyScript.on_server_message(line, clean_line)
end)

function _G.MyScript.on_server_message(line, clean_line)
    if not _G.MyScript.state.running then return end
    -- ... 訊息處理邏輯
end

-- ===== 公開 API =====
function _G.MyScript.start()
    _G.MyScript.state.running = true
    _G.MyScript.state.run_id = MudUtils.get_new_run_id()
    MudUtils.register_quest("MyScript", _G.MyScript.stop)
    MudUtils.start_log("myscript")
    mud.echo("[MyScript] 🚀 啟動")
end

function _G.MyScript.stop()
    _G.MyScript.state.running = false
    MudUtils.stop_log()
    mud.echo("[MyScript] 🛑 停止")
end

```

```
function _G.MyScript.reload()
    package.loaded["scripts.myscript"] = nil
    require("scripts.myscript")
    mud.echo("[MyScript] 🔄 已重新載入")
end

-- ===== 啟動說明 =====
MudUtils.print_script_help("MyScript", "v1.0", "描述", {
    { cmd = "MyScript.start()", desc = "啟動" },
    { cmd = "MyScript.stop()", desc = "停止" },
})
```

顯式依賴宣告（必要）

✗ 禁止：隱性依賴全域

```
-- 假設 _G.MudUtils 已存在 → 若載入順序異常，直接 nil 錯誤
MudUtils.register_hook("MyScript", ...)
```

✓ 正確：顯式 require

```
local MudUtils = require_module("MudUtils")
MudUtils.register_hook("MyScript", ...)
```

理由：

- `require` 有快取，重複呼叫不會重新執行，零效能損耗
- 依賴關係文字化，易於閱讀與維護
- 不依賴載入順序，腳本自給自足

模組介紹

模組	功能
MudUtils	Hook 管理、計時器、日誌、任務生命週期
MudNav	路徑行走、體力恢復、走路回呼
MudCombat	安全召喚 (safe_summon)、多次重試
MudExplorer	DFS/BFS 探索、目標搜尋
MudLoot	戰利品收集、自動 get
MudMapper	地圖記錄、廣度優先探索

Hook 機制

所有伺服器訊息統一由 `MudUtils` 的 dispatcher 分發。腳本必須透過 `register_hook` 訂閱，而非直接複寫 `_G.on_server_message`（複寫會覆蓋其他腳本的 hook）。

```
--  正確
MudUtils.register_hook("MyScript", function(line, clean_line)
    _G.MyScript.on_server_message(line, clean_line)
end)

--  禁止
_G.on_server_message = function(line, clean_line)
    -- 這會清除所有其他腳本的 hook !
end
```

重新載入後 hook 不疊加

`register_hook` 以名稱為 key，重複呼叫會覆蓋舊 hook，不會集狀疊加。這是 `reload()` 安全的原因。

全域命名空間規範

- 每個腳本使用唯一的大寫命名空間： `_G.MyScript`
- 初始化時使用 `or {}`，確保 reload 不清除現有狀態（除非刻意重置）

```
_G.MyScript = _G.MyScript or {}
_G.MyScript.state = _G.MyScript.state or { running = false }
```

Timer 與 run_id 防競態

腳本啟動時呼叫 `MudUtils.get_new_run_id()` 取得一個遞增的 ID。所有計時器回呼的第一件事就是驗證 `run_id`，確保舊計時器不會干擾新的執行週期。

```
function _G.MyScript.start()
    local rid = MudUtils.get_new_run_id()
    _G.MyScript.state.run_id = rid

    MudUtils.safe_timer(2.0, function(new_rid)
        if new_rid ~= _G.MyScript.state.run_id then return end -- 防舊 timer
        -- ... 執行邏輯
    end)
end
```

`MudUtils.safe_timer` 會自動封裝 `run_id`，推薦優先使用。

reload() 標準實作

```
function _G.MyScript.reload()
    package.loaded["scripts.myscript"] = nil -- 清除 require 快取
    require("scripts.myscript")              -- 重新執行腳本
    mud.echo("[MyScript] 🔄 已重新載入")
end
```

注意：模組腳本（ scripts/modules/ ）的 reload key 格式為 "scripts.modules.MudUtils" 。

腳本載入流程一覽



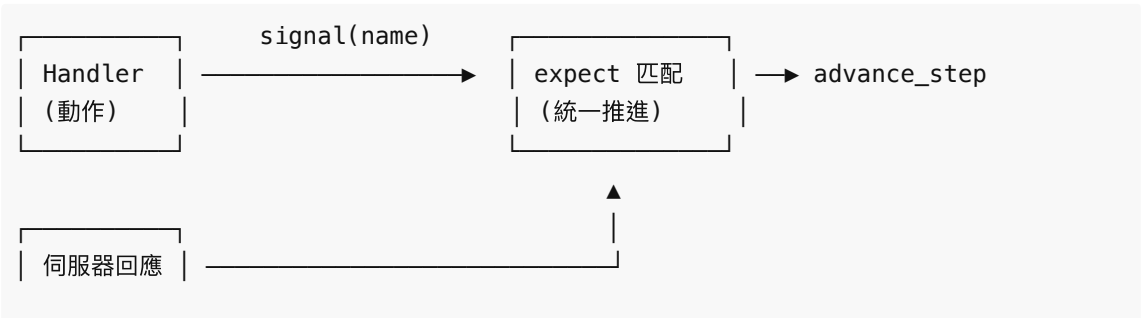
Modules 層只執行一次，後續的 require 直接命中快取（ package.loaded ），零開銷。

任務腳本架構（Signal Pattern）

任務腳本（如 smurf_quest.lua 、 poker_quest.lua ）建議採用 **Signal Pattern**，統一以 expect 匹配推進步驟，避免多重推進路徑造成的 race condition 。

核心原則

Handler 只管動作，**Expect** 統一管推進。



| (直接匹配) | 如: "賈不妙的城堡外"

signal 函數

Handler 完成時呼叫 `signal(name)` 注入虛擬訊號到 `on_server_message`，觸發 expect 匹配：

```
local function signal(name)
    MudUtils.safe_timer(0.3, function(rid)
        if MudUtils.check_run(rid) then
            _G.MyQuest.on_server_message("__SIGNAL__:" .. name)
        end
    end)
end
```

QUEST_STEPS 定義方式

```
local QUEST_STEPS = {
    -- 伺服器回應直接匹配
    {name="go_entrance",      cmds={"5n;2w;n"}, expect="目標房間名稱"},

    -- Handler 完成後發信號推進
    {name="summon_npc",      cmds={},
    expect="__SIGNAL__:summon_npc"},

    -- 發送指令後等伺服器回應
    {name="talk_npc",        cmds={"ta npc yes"}, expect="NPC 回應文字"},
}
```

on_server_message 推進邏輯

```
-- 統一推進：無條件匹配 expect 後 advance
local step = QUEST_STEPS[s.step_index]
if step and step.expect ~= "" and not s.step_completed then
    if clean_line:find(step.expect, 1, true) then
        s.step_completed = true
        MudUtils.safe_timer(0.5, advance_step)
    end
end
```

Handler 範例

```
-- 召喚類：等 MudCombat 驗證完畢再發信號
function step_handlers.summon_npc(rid)
```

```

MudCombat.safe_summon("NPC名", "c sum npc", {max_retries=10},
    function() signal("summon_npc") end,    -- 🟢 不呼叫 advance_step
    function() MyQuest.stop() end
)
end

-- 戰鬥類：handler 負責循環施技，死亡偵測寫在 on_server_message
function step_handlers.kill_boss(rid)
    mud.send("kill boss")
    local function loop(loop_rid)
        if not MudUtils.check_run(loop_rid) then return end
        if QUEST_STEPS[state.step_index].name ~= "kill_boss" then return
    end
        mud.send("skill boss")
        MudUtils.safe_timer(4.0, loop)
    end
    loop(rid)
end

-- on_server_message 中偵測死亡並發信號
if clean_line:find("Boss魂歸西天了", 1, true) then
    signal("kill_boss")
end

-- 搜刮類：等 MudLoot 完成再發信號
function step_handlers.get_loot(rid)
    MudUtils.safe_timer(1.0, function(new_rid) -- 等掉落結算
        MudLoot.process_loot({items={"item"}, fallback_blind=true},
            function() signal("get_loot") end
        )
    end)
end
end

```

常見陷阱

❌ 雙重推進 Race Condition

```

-- 反面教材：handler 和 expect 同時推進
{name="get_item", expect="拿出了 物品"}, -- expect 會匹配推進
function step_handlers.get_item()
    MudLoot.process_loot({...}, function()
        advance_step() -- handler 也推進 → 跳過下一步！
    end)
end
end

```

解法：使用 signal pattern，handler 永遠不直接呼叫 `advance_step()`。

✗ 移動力不足導致卡住

技能需要移動力（如大地之斬需 $mv > 500$ ），移動力耗盡時技能施放失敗但不會進入戰鬥。

解法：在 `on_server_message` 中偵測並自動恢復：

```
if clean_line:find("移動力不足") or clean_line:find("精疲力竭") then
    mud.send("c ref") -- 恢復移動力
end
```

同時戰鬥開局應加入普攻保底 `mud.send("kill target")`。

✗ 搜刮時機過早

怪物死亡後系統需要數行輸出結算（經驗值、金幣、掉落物）。立即 `look` 可能看不到屍體。

解法：搜刮步驟加入 1 秒延遲等待結算完成。

✗ Watchdog 誤殺長時間戰鬥

戰鬥可能超過 watchdog timeout（預設 180 秒），觸發自動停止。

解法：Watchdog 檢查 `MudCombat.is_fighting()` 並展延活躍時間：

```
if MudCombat.is_fighting() then
    update_activity()
end
```

✗ 使用 Prompt 作為偵測條件

Prompt（如（hp2741/2741 ma2154/2154 v1364/1364 ...））是玩家自訂的格式，不同玩家的 prompt 完全不同，甚至可能為空。永遠不要拿 prompt 的內容來判斷訊息結束、解析狀態或觸發邏輯。

```
-- ✗ 禁止
if clean_line:find("^%(hp%d") then -- prompt 格式因人而異！
    scan_complete = true
end

-- ✓ 正確：使用伺服器固定輸出標記
if clean_line:find("[出口:", 1, true) then
    scan_complete = true
end
```

✗ 行走途中遭遇非預期戰鬥

MudNav 走路時可能遇到主動攻擊的怪物（如野狗），導致移動指令被伺服器拒絕（不行！你現在正身陷戰鬥中！），後續路步全部錯位。

解法：在 `on_server_message` 中偵測並自動逃跑，逃跑後終止任務：

```
if step.name ~= "kill_boss" then -- 排除預期的戰鬥步驟
    if clean_line:find("你現在正身陷戰鬥中") then
        mud.send("flee")
    end
    if clean_line:find("不顧面子從戰鬥中逃了") then
        MyQuest.stop()
    end
end
```